

Semester Project: Flee The Maze

A Semester Project Report

Course: CSE142 Object Oriented Programming Techniques
Semester: Spring 2026
Instructor: Behraj Khan
Due Date: 10th May, 2026
Author(s): Ammar Qureshi — ERP: 33410
Arham Shah — ERP: 33459

Department of Computer Science
Institute of Business Administration, Karachi
May 10, 2026

Abstract

"Flee The Maze" is a first-person 3D horror game developed in C++ using Raylib. The player is placed inside a randomly generated maze and must find the exit while being hunted by AI-controlled ghosts. The game consists of three progressively larger and more difficult levels.

The problem addressed is the design and implementation of an interactive 3D game using object-oriented and algorithmic principles. The maze is generated procedurally using an iterative Depth First Search (DFS) backtracking algorithm built on a custom generic Stack data structure. Ghosts are stored in a custom generic Queue data structure with operator overloading, and they navigate the maze using the A* pathfinding algorithm with manhattan distance heuristic. The Ghost class inherits from an abstract base class Entity and overrides the CheckCollision() and Update() functions. The player is implemented with a First Person Perspective (FPP), also inheriting from the class Entity, and overriding the same functions.

The result is a fully playable horror game that demonstrates the applications and usage of data structure, algorithms, and OOP principles within a real-time interactive context.

Keywords: [Object-Oriented Programming, A* Pathfinding, DFS, C++, Inheritance, Polymorphism, Stack, Queue, Raylib, Game Development, Templates,...]

Contents

Abstract	2
1 Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Scope and Limitations	5
1.5 Initial Design Plan	5
2 OOP Design	6
2.1 Class Hierarchy	6
2.2 Class Descriptions	7
2.3 OOP Concepts Applied	8
3 Implementation	8
3.1 Development Environment	8
3.2 Base Class (Entity.hpp / .cpp)	8
3.3 Derived Classes (Player.hpp / Ghost.hpp)	9
3.4 Demonstrating Polymorphism	9
3.5 Data Structures	12
3.6 Additional Key Functions	15
4 Testing	18
4.1 Test Cases	18
4.2 Edge Cases	19
5 Results and Discussion	19
5.1 Results	19
5.2 Analysis	20
6 Challenges and Solutions	20
6.1 Ghost Pathfinding Through Walls	20
6.2 Ghost Freezing Near Player	21
7 Future Work	21
8 Conclusion	21
References	22
A Complete Source Code	23
A.1 main.cpp	23
A.2 Entity.hpp	23
A.3 Ghost.hpp	24
A.4 Player.hpp	24
A.5 Cell.hpp	25
A.6 Cell.cpp	25
A.7 Maze.hpp	26
A.8 Game.hpp	27
B Installation and Usage	28

B.1	Prerequisites	28
B.2	Setup and Compilation	28
B.3	Run the Game	28
B.4	Controls	29
B.5	Gameplay Instructions	29

1 Introduction

1.1 Background

Game Development requires the simultaneous management of rendering, AI, input, audio, and memory, making it an ideal domain for demonstrating OOP principles. "Flee The Maze" addresses procedural maze generation, ghost AI, and real-time game state management, each of which maps naturally to OOP concepts. The project applies core principles taught in CSE142 including inheritance, polymorphism, encapsulation, abstraction, templates, dynamic memory management, data structures, and operator overloading throughout the game loop.

1.2 Problem Statement

The problem is to design and implement a real-time 3D horror game in which a player navigates a randomly generated maze while being hunted by AI ghosts, with the maze size and ghost count increasing across three levels. This is challenging because it requires multiple complex systems including procedural maze generation, pathfinding AI, collision detection, and real-time rendering, to work correctly and simultaneously within a single game loop. The input is provided by the player each frame through the keyboard (WASD and SHIFT keys) and mouse, while the output is a rendered 3D scene with responsive movement, ghost behaviour, level progression, and game-state transitions. The constraints include ensuring that the maze is perfectly connected, with exactly one path between any two cells. Moreover, both the player and ghosts must navigate the maze without passing through walls, and all systems must run efficiently enough to maintain real-time performance as the maze grows larger each level.

1.3 Objectives

1. Design a class hierarchy that models the problem domain.
2. Implement core OOP concepts: encapsulation, inheritance, polymorphism, and abstraction.
3. Demonstrate correct use of constructors, destructors, and operator overloading.
4. Implement and apply custom generic data structures, including a templated Stack for iterative DFS maze generation and a templated Queue with operator overloading for ghost management.

1.4 Scope and Limitations

This project includes a fully playable 3D first-person horror game with three levels, procedurally generated mazes, AI ghosts, collision detection, and audio. It does not include multiplayer functionality, saved game states, or difficulty settings. Moreover, all asset files are assumed to be present in their correct directories at runtime, and the game runs at a fixed resolution of 1280x720.

1.5 Initial Design Plan

The following outlines the initial design plan formulated before implementation began.

Core Game Mechanics:

- First-person 3D maze navigation
- Randomly generated mazes

- Player spawns at a random entrance
- AI ghosts that chase the player
- Find the exit to progress to the next level
- Each level is progressively difficult than the last one

Planned Classes: Maze, Cell, Player, Ghost, Pathfinder, and a Game manager for level progression, difficulty scaling, and random spawns.

Planned Data Structures: A 2D vector for the maze grid, a generic Queue for ghost storage, and a generic Stack for DFS maze generation.

Planned Algorithms: Iterative DFS backtracking for maze generation, A* pathfinding with Manhattan distance heuristic for ghost AI, wall-based collision detection, a Ghost FSM with Patrol (if far), Chase (if nearby), and Attack states, and escape validation.

Deviations from Plan: The Pathfinder class was merged directly into Ghost. The FSM ghost behaviour was simplified to always-chase due to time constraints and FSM added unnecessary complexity to a three-level game. Escape validation was handled implicitly by DFS, which guarantees a connected maze.

2 OOP Design

2.1 Class Hierarchy

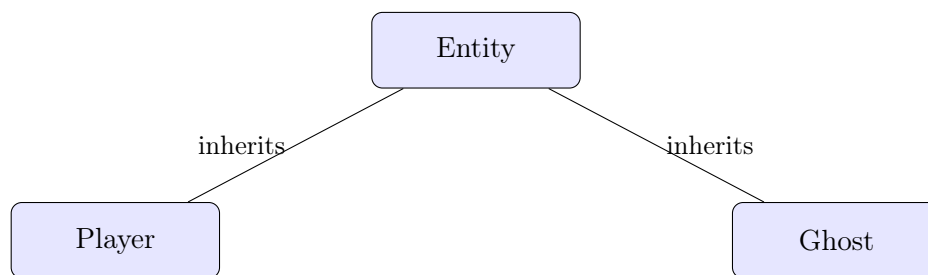


Figure 1: Class hierarchy diagram (is-a relationships)

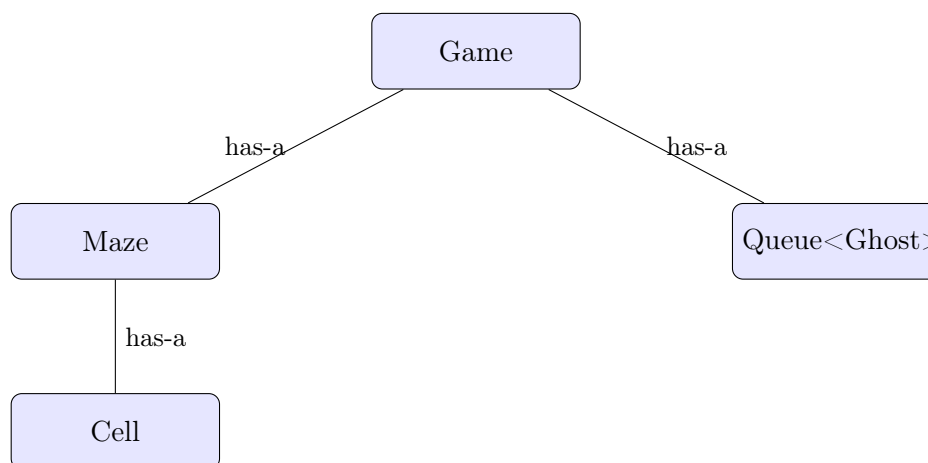


Figure 2: Class hierarchy diagram (has-a relationships)

The game is built around two types of relationships. For inheritance (is-a), the abstract base class **Entity** defines the common interface for all game objects through two pure virtual functions: `Update()` and `checkCollision()`. Both **Player** and **Ghost** inherit from **Entity** and provide their own concrete implementations of these functions. For composition (has-a), the **Game** class owns a pointer to a **Maze** object which is rebuilt each level, a **Player** object, and a `Queue<Ghost>` which stores all active ghost enemies. The **Maze** class internally owns a 2D vector of **Cell** objects, where each **Cell** tracks its four walls and visited state. The **Stack** and **Queue** classes are independent generic templated data structures used by **Maze** and **Game** respectively.

2.2 Class Descriptions

Table 1: Summary of Classes

Class	Responsibility	Key Members
Entity	Abstract base class for all game objects. Defines common interface through pure virtual functions.	<code>position</code> , <code>speed</code> , <code>Update()</code> , <code>checkCollision()</code>
Player	Inherits from Entity . Handles first-person camera, WASD movement, sprinting, and wall collision.	<code>camera</code> , <code>Update()</code> , <code>checkCollision()</code>
Ghost	Inherits from Entity . Uses A* pathfinding to chase the player. Stores path as vector of cell coords.	<code>path</code> , <code>target</code> , <code>aStar()</code> , <code>move()</code> , <code>isCaught()</code>
Game	Central game manager. Owns all objects, handles game states, rendering, audio, and level progression.	<code>*maze</code> , <code>player</code> , <code>ghosts</code> , <code>Update()</code> , <code>Draw()</code>
Maze	Generates maze using iterative DFS. Owns a 2D vector of Cells . Handles 3D drawing.	<code>maze[][]</code> , <code>generateMaze()</code> , <code>Draw3D()</code>
Cell	Represents a single maze cell. Tracks four walls and visited state.	<code>walls[]</code> , <code>visited</code> , <code>hasWall()</code> , <code>removeWall()</code>
Stack<T>	Generic templated stack using dynamic array. Used for DFS maze generation.	<code>push()</code> , <code>pop()</code> , <code>top()</code> , <code>isEmpty()</code>
Queue<T>	Generic templated linked-list queue. Used for ghost storage with operator overloading.	<code>enqueue()</code> , <code>dequeue()</code> , <code>operator+=</code> , <code>operator[]</code>

2.3 OOP Concepts Applied

- **Encapsulation:** All class member variables are private or protected, accessible only through public getters and setters. For example, `Entity` exposes `getPosition()` and `setPosition()` rather than allowing direct access to `position`. The `Maze` class encapsulates its internal `vector<vector<Cell>>` and exposes only necessary operations like `getCell()`, `isValid()`, and `worldToCell()`.
- **Inheritance:** `Player` and `Ghost` both inherit from the abstract base class `Entity`, which defines the shared interface through pure virtual functions `Update()` and `checkCollision()`. This allows both classes to share common attributes like `position` and `speed` while providing their own specialized behaviour.
- **Polymorphism:** The pure virtual functions `Update()` and `checkCollision()` in `Entity` are overridden by both `Player` and `Ghost`. This allows the `Game` class to treat both as `Entity` objects through a common interface while each executes its own specific logic at runtime.
- **Operator Overloading:** The `Queue` class overloads several operators: `+=` to merge two ghost queues during level transitions, `[]` for index-based ghost access, `==` and `!=` to compare queue sizes, and `>` to detect hard mode when ghost count exceeds a threshold.
- **Templates:** Both `Stack` and `Queue` are implemented as generic templated classes using `template<typename T>`. `Stack<std::pair<int,int>>` is used for maze generation and `Queue<Ghost>` is used for ghost storage, demonstrating reusable type-independent data structures.

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Compiler:** g++ (MinGW-w64)
- **IDE:** Visual Studio Code
- **Build system:** Makefile
- **Platform:** Windows 11
- **Graphics Library:** Raylib 5.5

3.2 Base Class (Entity.hpp / .cpp)

```
1 class Entity {
2     protected:
3         Vector3 position;
4         float speed;
5     public:
6         Entity(Vector3 spawn = {0, 0, 0}) : position(spawn), speed
            (0.4f) {}
7         virtual void Update(const Maze* maze) = 0;
8         Vector3 getPosition() const { return position; }
9         void setPosition(Vector3 pos) { position = pos; }
10        virtual bool checkCollision(const Maze& maze, Vector3 pos)
            const = 0;
11        virtual ~Entity() {}
```



```
12 };
```

Listing 1: Base class definition and implementation

3.3 Derived Classes (Player.hpp / Ghost.hpp)

```
1 class Player : public Entity {
2     Camera3D camera; //fpp mode
3     float shift_speed; //booster
4     float yaw, pitch; //for rotation
5     Vector3 forward;
6     Vector3 right;
7     public:
8         Player(Vector3 spawn);
9         Player();
10        void Update(const Maze* maze) override;
11        Camera3D getCamera() const;
12        bool checkCollision(const Maze& maze, Vector3 pos) const
            override;
13 };
14 class Ghost : public Entity {
15     std::vector<std::pair<int, int>> path; //cell coords
16     std::pair<int, int> target; //current player cell
17     Vector3 targetPos; //current player pos
18     float repathTime, repathDelay;
19     std::vector<std::pair<int, int>> aStar(
20         const Maze& maze,
21         std::pair<int, int> start,
22         std::pair<int, int> end
23     ); //pathfinding
24     float manhattan(std::pair<int, int> a, std::pair<int, int> b)
25         const;
26     void move(const Maze* maze, Vector3 pp);
27     public:
28         Ghost(Vector3 spawn);
29         Ghost(); //position can be set through inherited setPos func
30         void setTarget(std::pair<int, int> tar, Vector3 pos);
31         void Update(const Maze* maze) override;
32         bool checkCollision(const Maze& maze, Vector3 pos) const
33             override;
34         bool isCaught(Vector3 tarPos) const;
35         void Draw(Texture2D mod, Camera3D cam) const;
36         void setRepathDelay(float rd);
37 };
```

Listing 2: Derived class with overridden virtual functions

3.4 Demonstrating Polymorphism

In this project, polymorphism is demonstrated through the abstract base class `Entity`, which declares `Update()` and `checkCollision()` as pure virtual functions. Both `Player` and `Ghost` inherit from `Entity` and provide their own concrete implementations. The `Game` class calls `Update()` on both through their respective objects, and each executes its own logic at runtime.

```
1 void Player::Update(const Maze* maze) {
2     float sensitivity = 0.005f;
3     Vector2 md = GetMouseDelta();
```

```
4     yaw -= md.x * sensitivity;
5     pitch -= md.y * sensitivity;
6     if (pitch>1.5f) pitch = 1.5f;
7     if (pitch<-1.5f) pitch = -1.5f;
8     Vector3 direc;
9     direc.x = cosf(pitch) * sinf(yaw);
10    direc.y = sinf(pitch);
11    direc.z = cosf(pitch) * cosf(yaw);
12    camera.target = {
13        camera.position.x + direc.x,
14        camera.position.y + direc.y,
15        camera.position.z + direc.z
16    };
17    forward.x = direc.x;
18    forward.y = 0.0f;
19    forward.z = direc.z;
20    float len = sqrtf(forward.x*forward.x + forward.y*forward.y +
21        forward.z*forward.z);
22    if (len!=0) {
23        forward.x /= len;
24        forward.y /= len;
25        forward.z /= len;
26    }
27    right.x = forward.y * camera.up.z - forward.z * camera.up.y;
28    right.y = forward.z * camera.up.x - forward.x * camera.up.z;
29    right.z = forward.x * camera.up.y - forward.y * camera.up.x;
30    float len2 = sqrtf(right.x*right.x + right.y*right.y + right.z*
31        right.z);
32    if (len2!=0) {
33        right.x /= len2;
34        right.y /= len2;
35        right.z /= len2;
36    }
37    speed = 0.03f;
38    if (IsKeyDown(KEY_LEFT_SHIFT) || IsKeyDown(KEY_RIGHT_SHIFT))
39        speed *= shift_speed;
40    if (IsKeyDown(KEY_W)) {
41        Vector3 pos = camera.position;
42        pos.x += forward.x * speed;
43        pos.z += forward.z * speed;
44        if (!checkCollision(*maze, pos)) {
45            camera.position = pos;
46            position = pos;
47        }
48    }
49    if (IsKeyDown(KEY_S)) {
50        Vector3 pos = camera.position;
51        pos.x -= forward.x * speed;
52        pos.z -= forward.z * speed;
53        if (!checkCollision(*maze, pos)) {
54            camera.position = pos;
55            position = pos;
56        }
57    }
58    if (IsKeyDown(KEY_D)) {
59        Vector3 pos = camera.position;
60        pos.x += right.x * speed;
61        pos.z += right.z * speed;
```

```

59         if (!checkCollision(*maze, pos)) {
60             camera.position = pos;
61             position = pos;
62         }
63     }
64     if (IsKeyDown(KEY_A)) {
65         Vector3 pos = camera.position;
66         pos.x -= right.x * speed;
67         pos.z -= right.z * speed;
68         if (!checkCollision(*maze, pos)) {
69             camera.position = pos;
70             position = pos;
71         }
72     }
73 }
74 bool Player::checkCollision(const Maze& maze, Vector3 pos) const {
75     float cells = maze.getCellSize(); //cell size
76     int col = (int)(pos.x/cells);
77     int row = (int)(pos.z/cells);
78     if (row<0 || row>=maze.getRow() || col<0 || col>=maze.getColumn()
79         ) return true;
80     const Cell& cell = maze.getCell(row, col);
81     float ox = fmod(pos.x, cells); //offset X
82     float oz = fmod(pos.z, cells); //offset Z
83     float ps = 0.2f; //player size
84     if (cell.hasWall(0) && oz<ps) return true;
85     if (cell.hasWall(1) && ox>(cells-ps)) return true;
86     if (cell.hasWall(2) && oz>(cells-ps)) return true;
87     if (cell.hasWall(3) && ox<ps) return true;
88     return false;
89 }
90 void Ghost::Update(const Maze* maze) {
91     repathTime += GetFrameTime();
92     auto curr = maze->worldToCell(position);
93     if (path.empty() || repathTime>=repathDelay) {
94         path = aStar(*maze, curr, target);
95         repathTime = 0.0f;
96     }
97     move(maze, targetPos);
98 }
99 bool Ghost::checkCollision(const Maze& maze, Vector3 pos) const {
100     float cs = maze.getCellSize();
101     int r = (int)(pos.z / cs);
102     int c = (int)(pos.x / cs);
103     if (!maze.isValid(r, c)) return true;
104     int cr = (int)(position.z / cs);
105     int cc = (int)(position.x / cs);
106     if (!maze.isValid(cr, cc)) return true;
107     int dr = r - cr;
108     int dc = c - cc;
109     const Cell& ccell = maze.getCell(cr, cc);
110     if (dr==1 && dc==0) return ccell.hasWall(2);
111     if (dr==-1 && dc==0) return ccell.hasWall(0);
112     if (dr==0 && dc==1) return ccell.hasWall(1);
113     if (dr==0 && dc==-1) return ccell.hasWall(3);
114     return false;
115 }

```

Listing 3: Pure virtual functions in Entity overridden by Player and Ghost

3.5 Data Structures

This project implements two custom generic data structures. The `Stack<T>` is an array-based templated stack with auto-resize, used exclusively for the iterative DFS backtracking algorithm in `Maze::generateMaze()`. The `Queue<T>` is a linked-list based templated queue used to store and manage all active `Ghost` objects in the `Game` class. The `Queue` implements operator overloading including `+=` for merging ghost queues between levels, `[]` for index-based access, and `>` for hard mode detection, as well as a custom iterator to support range-based for loops.

```

1  template <typename T>
2  class Stack {
3      T* arr;
4      int capacity;
5      int size;
6      int topIdx;
7      void resize(int s) {
8          T* temp = new T[s];
9          for (int i=0;i<capacity;i++) temp[i] = arr[i];
10         delete[] arr;
11         capacity = s;
12         arr = temp;
13     }
14     public:
15         Stack(int s = 10) {
16             capacity = s;
17             size = 0;
18             arr = new T[capacity];
19             topIdx = -1;
20         }
21         void push(const T& val) {
22             if (isFull()) resize(capacity * 2);
23             size++;
24             arr[++topIdx] = val;
25         }
26         T pop() {
27             if (isEmpty()) throw std::underflow_error("Stack
28                 underflow");
29             size--;
30             return arr[topIdx--];
31         }
32         T& top() const {
33             if (isEmpty()) throw std::underflow_error("Stack
34                 underflow");
35             return arr[topIdx];
36         }
37         int getSize() const { return size; }
38         bool isEmpty() const { return size == 0; }
39         bool isFull() const { return size == capacity; }
40         ~Stack() {
41             delete[] arr;
42         }
43     };

```

Listing 4: Stack data structure used for DFS maze generation

```
1  template <typename T>
2  class Queue {
3      struct node {
4          T value;
5          node* next;
6          node(T val) : value(val), next(nullptr) {}
7      };
8      node* front;
9      node* rear;
10     int size;
11     public:
12         Queue() : size(0) {
13             front = rear = nullptr;
14         }
15         Queue(const Queue<T>& q) : size(0) {
16             front = rear = nullptr;
17             node* temp = q.front;
18             while (temp) {
19                 enqueue(temp->value);
20                 temp = temp->next;
21             }
22         }
23         Queue<T>& operator = (const Queue<T>& q) {
24             if (this==&q) return *this;
25             clear();
26             node* temp = q.front;
27             while (temp) {
28                 enqueue(temp->value);
29                 temp = temp->next;
30             }
31             return *this;
32         }
33         bool isEmpty() const {
34             return size == 0;
35         }
36         void enqueue(T val) {
37             node* temp = new node{val};
38             if (rear==nullptr) front = rear = temp;
39             else {
40                 rear->next = temp;
41                 rear = temp;
42             }
43             size++;
44         }
45         T dequeue() {
46             if (isEmpty()) throw std::runtime_error("Queue is empty!");
47             node* temp = front;
48             T val = temp->value;
49             front = front->next;
50             if (front==nullptr) rear = nullptr;
51             delete temp;
52             size--;
53             return val;
54         }
55         T& peek() {
56             if (isEmpty()) throw std::runtime_error("Queue is empty!");
```

```

    );
    return front->value;
}
const T& peek() const {
    if (isEmpty()) throw std::runtime_error("Queue is empty!");
    return front->value;
}
int getSize() const {
    return size;
}
void clear() {
    while (!isEmpty()) dequeue();
}

T& operator [](int idx) {
    if (idx<0 || idx>=size) throw std::out_of_range("Index
        out of bounds!");
    node* temp = front;
    for (int i=0;i<idx;i++) temp = temp->next;
    return temp->value;
}
const T& operator [](int idx) const {
    if (idx<0 || idx>=size) throw std::out_of_range("Index
        out of bounds!");
    node* temp = front;
    for (int i=0;i<idx;i++) temp = temp->next;
    return temp->value;
}
Queue<T>& operator += (const Queue<T>& q) {
    node* temp = q.front;
    while (temp) {
        enqueue(temp->value);
        temp = temp->next;
    }
    return *this;
}
//comps based on size
bool operator == (const Queue<T>& q) const { return size == q
    .size; }
bool operator != (const Queue<T>& q) const { return size != q
    .size; }
bool operator > (const Queue<T>& q) const { return size > q.
    size; }

//iterators
struct it {
    node* temp;
    it(node* n) : temp(n) {}
    T& operator * () { return temp->value; }
    T* operator -> () { return &temp->value;}
    it& operator ++ () {
        temp = temp->next;
        return *this;
    }
    bool operator != (const it& i) const { return temp != i.
        temp; }
};

```

```

107     struct const_it {
108         node* temp;
109         const_it(node* n) : temp(n) {}
110         const T& operator * () const { return temp->value; }
111         const T* operator -> () const { return &temp->value; }
112         const_it& operator ++ () {
113             temp = temp->next;
114             return *this;
115         }
116         bool operator != (const const_it& i) const { return temp
            != i.temp; }
117     };
118     it begin() { return it(front); }
119     it end()   { return it(nullptr); }
120     const_it begin() const { return const_it(front); }
121     const_it end() const { return const_it(nullptr); }
122
123     ~Queue() {
124         clear();
125     }
126 };

```

Listing 5: Queue data structure used for ghost storage with operator overloading

3.6 Additional Key Functions

The following key functions demonstrate important implementation details of this project:

```

1  std::vector<std::pair<int, int>> Ghost::aStar(
2      const Maze& maze,
3      std::pair<int, int> start,
4      std::pair<int, int> end
5  ) {
6      std::map<std::pair<int, int>, bool> inOpen, inClosed;
7      std::vector<std::pair<int, int>> open;
8      std::map<std::pair<int, int>, std::pair<int, int>> parent;
9      std::map<std::pair<int, int>, float> gScore;
10     open.push_back(start);
11     inOpen[start] = true;
12     gScore[start] = 0.0f;
13     std::vector<std::pair<int, int>> dir = {
14         {1, 0}, {-1, 0},
15         {0, 1}, {0, -1}
16     };
17     while (!open.empty()) {
18         int bestIdx = 0;
19         float bestF = gScore[open[0]] + manhattan(open[0], end);
20         for (size_t i=1; i<open.size();i++) {
21             float f = gScore[open[i]] + manhattan(open[i], end);
22             if (f<bestF) {
23                 bestF = f;
24                 bestIdx = (int)i;
25             }
26         }
27         auto curr = open[bestIdx];
28         open.erase(open.begin() + bestIdx);
29         if (curr==end) {
30             std::vector<std::pair<int, int>> res;

```

```

31         auto step = end;
32         while (parent.count(step)) {
33             res.push_back(step);
34             step = parent[step];
35         }
36         std::reverse(res.begin(), res.end());
37         return res;
38     }
39     inClosed[curr] = true;
40     for (auto& x : dir) {
41         int r = curr.first + x.first;
42         int c = curr.second + x.second;
43         std::pair<int, int> next = {r, c};
44         if (!maze.isValid(r, c)) continue;
45         if (inClosed.count(next)) continue;
46         float cs = maze.getCellSize();
47         Vector3 np = {
48             c * cs + cs/2.0f,
49             position.y,
50             r * cs + cs/2.0f
51         };
52         if (checkCollision(maze, np)) continue;
53         float newG = gScore[curr] + 1.0f;
54         if (!gScore.count(next) || newG < gScore[next]) {
55             gScore[next] = newG;
56             parent[next] = curr;
57             if (!inOpen.count(next)) {
58                 open.push_back(next);
59                 inOpen[next] = true;
60             }
61         }
62     }
63 }
64 return {};
65 }

```

Listing 6: A* pathfinding algorithm in Ghost

```

1 void Maze::generateMaze() {
2     Stack<std::pair<int, int>> s;
3     int rs = 0, rc = 0;
4     maze[rs][rc].setVisited(true);
5     s.push({rs, rc});
6     while (!s.isEmpty()) {
7         auto [r, c] = s.top();
8         std::vector<std::pair<int, int>> neighbors;
9         std::vector<int> direc;
10        for (int i=0; i<4; i++) {
11            int nr = r + dirR[i];
12            int nc = c + dirC[i];
13            if (isValid(nr, nc) && !maze[nr][nc].isVisited()) {
14                neighbors.push_back({nr, nc});
15                direc.push_back(i);
16            }
17        }
18        if (!neighbors.empty()) {
19            int idx = std::rand() % neighbors.size();
20            int nr = neighbors[idx].first;

```



```
21         int nc = neighbors[idx].second;
22         int dir = direc[idx];
23         if (dir<0 || dir>3) throw std::out_of_range("Invalid
           direction in maze!");
24         maze[r][c].removeWall(dir);
25         maze[nr][nc].removeWall(getOppositeWall(dir));
26         maze[nr][nc].setVisited(true);
27         s.push({nr, nc});
28     }
29     else s.pop();
30 }
31 int entry = std::rand() % 4;
32 int exit;
33 do {
34     exit = std::rand() % 4;
35 } while (exit == entry);
36 if (entry==0) {
37     entryRow = 0;
38     entryColumn = std::rand() % columns;
39     Entrance = {entryRow, entryColumn};
40     maze[entryRow][entryColumn].removeWall(0);
41 }
42 else if (entry==1) {
43     entryRow = std::rand() % rows;
44     entryColumn = columns - 1;
45     Entrance = {entryRow, entryColumn};
46     maze[entryRow][entryColumn].removeWall(1);
47 }
48 else if (entry==2) {
49     entryRow = rows - 1;
50     entryColumn = std::rand() % columns;
51     Entrance = {entryRow, entryColumn};
52     maze[entryRow][entryColumn].removeWall(2);
53 }
54 else {
55     entryRow = std::rand() % rows;
56     entryColumn = 0;
57     Entrance = {entryRow, entryColumn};
58     maze[entryRow][entryColumn].removeWall(3);
59 }
60 if (exit==0) {
61     exitRow = 0;
62     exitColumn = std::rand() % columns;
63     Exit = {exitRow, exitColumn};
64     maze[exitRow][exitColumn].removeWall(0);
65 }
66 else if (exit==1) {
67     exitRow = std::rand() % rows;
68     exitColumn = columns - 1;
69     Exit = {exitRow, exitColumn};
70     maze[exitRow][exitColumn].removeWall(1);
71 }
72 else if (exit==2) {
73     exitRow = rows - 1;
74     exitColumn = std::rand() % columns;
75     Exit = {exitRow, exitColumn};
76     maze[exitRow][exitColumn].removeWall(2);
77 }
```

```

78     else {
79         exitRow = std::rand() % rows;
80         exitColumn = 0;
81         Exit = {exitRow, exitColumn};
82         maze[exitRow][exitColumn].removeWall(3);
83     }
84 }

```

Listing 7: DFS maze generation using custom Stack

4 Testing

4.1 Test Cases

Table 2: Test Cases and Results

TC #	Input / Action	Expected Output	Actual Output	Pass?
TC-01	Game launches	3D maze renders with player at entrance	As expected	✓
TC-02	Player reaches the exit cell	Winner sound plays, loading screen appears	As expected	✓
TC-03	Ghost catches the player	Jumpscare sound, red screen, level restarts	As expected	✓
TC-04	Player completes level 3	Gold celebration screen appears, game resets	As expected	✓
TC-05	Ghost pathfinds through maze	Ghost navigates only through open passages	As expected	✓
TC-06	Player completes a level	Maze size increases by 5, ghost count increases by 2	As expected	✓

4.2 Edge Cases

Table 3: Edge Cases and Handling

Edge Case	Description	How Handled
Ghost spawns too close to player	Ghost could spawn directly on top of player at level start	Spawn coordinates re-generated until ghost is at least 3 cells from entrance
Ghost and player in same cell	A* returned empty path causing ghost to freeze	Removed early exit from A*, ghost moves directly toward player world position
Maze floor disappearing on level up	Grass plane was sized for level 1 only	Floor plane and star positions recalculated each level to match new maze size
Ghost pathfinding through walls	Positional collision check failed at cell centres	Replaced with direct <code>hasWall()</code> checks in A* neighbour expansion
Dequeue on empty Queue	Calling dequeue on empty queue causes undefined behaviour	Queue throws <code>std::runtime_error</code> if dequeue or peek called on empty queue
Stack capacity exceeded during DFS	On large mazes the DFS stack could exceed initial capacity of 10	Stack automatically re-sizes to double capacity when full, preventing any fixed-size limitation
Stack underflow on empty stack	Calling <code>pop()</code> or <code>top()</code> on an empty stack causes undefined behaviour	Stack throws <code>std::underflow_error</code> if <code>pop()</code> or <code>top()</code> is called when stack is empty
Exit spawning on same coordinates as entrance	Exit could be generated at the same position as the entrance, making the maze unsolvable	Exit coordinates are re-generated in a <code>do-while</code> loop until the exit side differs from the entrance side

5 Results and Discussion

5.1 Results

The game successfully implements all core features across three levels. The following table summarises the performance characteristics of the key operations:

Table 4: Performance / Benchmark Results

Note: Timing values are estimated based on algorithm complexity and could differ in real-time. Frame rate measured on the development machine.

Operation	Input Size	Time (ms)	Notes
Maze Generation (DFS)	5x5	<1	Level 1
Maze Generation (DFS)	10x10	<1	Level 2
Maze Generation (DFS)	15x15	<1	Level 3
A* Pathfinding	5x5	<1	1 ghost
A* Pathfinding	10x10	<2	3 ghosts
A* Pathfinding	15x15	<5	5 ghosts
Frame Rate	All levels	60 fps	Stable on Intel UHD 620

5.2 Analysis

Design Objectives: All core design objectives were met. The class hierarchy with **Entity** as an abstract base class was successfully implemented with **Player** and **Ghost** as derived classes. Custom generic **Stack** and **Queue** data structures were implemented and actively used in maze generation and ghost management respectively. The game runs across three levels with increasing maze size and ghost count, and all game states including **PLAYING**, **JUMPSCARE**, **LOADING**, and **CLEAR** function correctly.

Complexity Analysis:

- **Maze Generation (DFS):** $O(r \times c)$ where r and c are rows and columns. Every cell is visited exactly once. Stack operations are $O(1)$ on average with auto-resize. Matches theoretical expectation.
- **A* Pathfinding:** $O(n^2 \log n)$ in our implementation; the open list is a **vector** requiring $O(n)$ linear scan per iteration to find the minimum f value, and each **map** operation (for **gScore**, **parent**, **inOpen**, **inClosed**) adds an $O(\log n)$ factor. Theoretical A* with a binary heap priority queue would be $O(n \log n)$.
- **Queue operations:** **enqueue** and **dequeue** are $O(1)$. Index access via **operator[]** is $O(n)$ due to linked list traversal.

Unexpected Behaviour and Limitations:

- Ghost movement occasionally froze when the ghost and player occupied the same cell, resolved by removing the early exit from A* and moving directly toward the player's world position.
- The grass floor disappeared on higher levels due to the plane being sized only at initialization, resolved by rebuilding the plane each level.
- Performance degrades slightly on level 3 with 5 ghosts each repathing every 0.8 seconds on a 15x15 maze, due to the $O(n^2)$ A* implementation.

6 Challenges and Solutions

6.1 Ghost Pathfinding Through Walls

Problem: Ghosts were passing through maze walls because the A* algorithm checked collisions using world positions, and cell centres are always far from any wall, so the check never triggered.

Solution: Instead of checking world positions, we now directly check whether a wall exists

between two cells using `hasWall()` before allowing A* to expand its neighbour.

Learning: In grid-based games, collision should be checked using the grid's own wall data rather than geometric distances.

6.2 Ghost Freezing Near Player

Problem: When the ghost reached the same cell as the player, A* returned an empty path and the ghost stopped moving completely.

Solution: Removed the early exit `if (start==end) return {};` from A*, and made the ghost move directly toward the player's exact world position when no path exists.

Learning: Cell-based pathfinding is too rough for the final approach. When two objects are in the same cell, direct world-position movement is needed to close the remaining distance.

7 Future Work

Possible extensions and improvements:

- Add a menu screen with Play, Tutorial, and Exit buttons for a more complete user experience.
- Improve lighting using Raylib shaders to create a torch effect, limiting visibility and increasing the horror atmosphere.
- Differentiate the entrance and exit using distinct icons or markers so the player can identify them at a glance.
- Introduce collectible objectives that must be completed before the exit unlocks, adding an exploration objective.
- Implement a lives system so the player has multiple chances before a full game over, rather than restarting the level on every catch.

8 Conclusion

"Flee The Maze" successfully demonstrates the practical application of OOP principles in the development of a real-time 3D horror game. The project addressed the challenges of procedural maze generation, ghost AI, and real-time game state management using a well-structured class hierarchy built around an abstract `Entity` base class with `Player` and `Ghost` as derived classes.

The custom generic `Stack` and `Queue` data structures were implemented from scratch and actively used throughout the project; the Stack for iterative DFS maze generation and the Queue for ghost storage with operator overloading. The A* pathfinding algorithm with Manhattan distance heuristic enables ghosts to navigate the maze, and all three levels function correctly with increasing difficulty.

The most valuable learning outcomes were that collision detection in grid-based games should use wall flags rather than world positions, that managing multiple systems like rendering, AI, and audio simultaneously within a single game loop is harder than it seems and that OOP principles like inheritance, encapsulation, polymorphism, and abstraction make complex projects significantly easier to organize and extend.

References

List references in IEEE format. Example entries:

- [1] *cppreference.com*. [Online]. Available: <https://en.cppreference.com>. [Accessed: May 10, 2026].
- [2] Raylib. *Raylib Documentation*. [Online]. Available: <https://www.raylib.com>. [Accessed: May 10, 2026].
- [3] A. Qureshi [asqtecki], *Flee The Maze — Source Code*. GitHub. [Online]. Available: <https://github.com/asqtecki/Flee-The-Maze>. [Accessed: May 10, 2026].
- [4] *Introduction to Algorithms*, T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, 4th ed. MIT Press, 2022.

A Complete Source Code

Full source code is available on GitHub at: <https://github.com/asqtecki/Flee-The-Maze>
Few code snippets are pasted below, while others can be found at the given URL of my GitHub repository.

A.1 main.cpp

```
1  #include <raylib.h>
2  #include <iostream>
3  #include "include/Game.hpp"
4  #include <cstdlib>
5  using namespace std;
6
7  struct Screen {
8      const int width = 1280;
9      const int height = 720;
10 };
11
12 int main() {
13     srand(time(0));
14     Screen s;
15     InitWindow(s.width, s.height, "Flee the Maze!");
16     DisableCursor();
17     SetTargetFPS(60);
18     Texture2D wall = LoadTexture("Graphics/wall.jpg");
19     Mesh cube = GenMeshCube(1.0f, 1.0f, 1.0f);
20     Model wallModel = LoadModelFromMesh(cube);
21     wallModel.materials[0].maps[MATERIAL_MAP_DIFFUSE].texture = wall;
22     Game g(wallModel);
23     while (!WindowShouldClose()) {
24         g.Update();
25         g.Draw();
26     }
27     UnloadModel(wallModel);
28     UnloadTexture(wall);
29     CloseWindow();
30 }
```

Listing 8: main.cpp — the driver

A.2 Entity.hpp

```
1  #pragma once
2
3  #include <raylib.h>
4  #include "Maze.hpp"
5
6  class Entity {
7      protected:
8          Vector3 position;
9          float speed;
10     public:
11         Entity(Vector3 spawn = {0, 0, 0}) : position(spawn), speed(0.4f) {}
12         virtual void Update(const Maze* maze) = 0;
13         Vector3 getPosition() const { return position; }
```

```

14     void setPosition(Vector3 pos) { position = pos; }
15     virtual bool checkCollision(const Maze& maze, Vector3 pos)
16         const = 0;
17     virtual ~Entity() {}
18 };

```

Listing 9: Entity.hpp — abstract base class

A.3 Ghost.hpp

```

1  #pragma once
2
3  #include "Entity.hpp"
4  #include "Maze.hpp"
5  #include <vector>
6  #include <utility>
7
8  class Ghost : public Entity {
9      std::vector<std::pair<int, int>> path; //cell coords
10     std::pair<int, int> target; //current player cell
11     Vector3 targetPos; //current player pos
12     float repathTime, repathDelay;
13     std::vector<std::pair<int, int>> aStar(
14         const Maze& maze,
15         std::pair<int, int> start,
16         std::pair<int, int> end
17     ); //pathfinding
18     float manhattan(std::pair<int, int> a, std::pair<int, int> b)
19         const;
20     void move(const Maze* maze, Vector3 pp);
21     public:
22         Ghost(Vector3 spawn);
23         Ghost(); //position can be set through inherited setPos func
24         void setTarget(std::pair<int, int> tar, Vector3 pos);
25         void Update(const Maze* maze) override;
26         bool checkCollision(const Maze& maze, Vector3 pos) const
27             override;
28         bool isCaught(Vector3 tarPos) const;
29         void Draw(Texture2D mod, Camera3D cam) const;
30         void setRepathDelay(float rd);
31 };

```

Listing 10: Ghost.hpp — derived class with A* pathfinding

A.4 Player.hpp

```

1  #pragma once
2  #include <raylib.h>
3  #include "Maze.hpp"
4  #include "Entity.hpp"
5
6  class Player : public Entity {
7      Camera3D camera; //fpp mode
8      float shift_speed; //booster
9      float yaw, pitch; //for rotation
10     Vector3 forward;
11     Vector3 right;

```



```

12     public:
13         Player(Vector3 spawn);
14         Player();
15         void Update(const Maze* maze) override;
16         Camera3D getCamera() const;
17         bool checkCollision(const Maze& maze, Vector3 pos) const
18             override;
19     };

```

Listing 11: Player.hpp — Derived class for the main character

A.5 Cell.hpp

```

1  //Represent single grid unit in Maze
2  //Stores position
3  //Tracks wether a unit has been visited or not, for maze generation
4
5  #pragma once
6
7  class Cell {
8      int rows, columns;
9      bool visited;
10     bool walls[4]; //0 = top/north, 1 = right/east, 2 = bottom/south,
11                  //3 = left/west
12     public:
13         Cell();
14         Cell(int r, int c);
15         int getRow() const;
16         int getColumn() const;
17         bool isVisited() const; //checks if unit has been visited
18         void setVisited(bool v);
19         bool hasWall(int idx) const; //checks if a wall exists
20         void removeWall(int idx); //removes a certain wall
21         void setWall(int idx, bool v); //sets wall state
22     };

```

Listing 12: Cell.hpp – Represents a unit within the maze

A.6 Cell.cpp

```

1  #include "../include/Cell.hpp"
2  #include <stdexcept>
3
4  Cell::Cell() : rows(0), columns(0), visited(false) {
5      for (int i=0;i<4;i++) walls[i] = true;
6  }
7
8  Cell::Cell(int r, int c) : rows(r), columns(c), visited(false) {
9      for (int i=0;i<4;i++) walls[i] = true;
10 }
11
12 int Cell::getRow() const {
13     return rows;
14 }
15
16 int Cell::getColumn() const {
17     return columns;
18 }

```

```

18 }
19
20 bool Cell::isVisited() const {
21     return visited;
22 }
23
24 void Cell::setVisited(bool v) {
25     visited = v;
26 }
27
28 bool Cell::hasWall(int idx) const {
29     if (idx >= 0 && idx < 4) return walls[idx];
30     throw std::out_of_range("Index out of bounds!");
31 }
32
33 void Cell::removeWall(int idx) {
34     if (idx >= 0 && idx < 4) {
35         walls[idx] = false;
36         return;
37     }
38     throw std::out_of_range("Index out of bounds!");
39 }
40
41 void Cell::setWall(int idx, bool v) {
42     if (idx >= 0 && idx < 4) {
43         walls[idx] = v;
44         return;
45     }
46     throw std::out_of_range("Index out of bounds!");
47 }

```

Listing 13: Cell.cpp — Cell class definition

A.7 Maze.hpp

```

1  #pragma once
2
3  #include <vector>
4  #include "Cell.hpp"
5  #include "Stack.hpp"
6  #include <utility>
7  #include <cstdlib>
8  #include <raylib.h>
9
10 class Maze {
11     int rows, columns; //height and width respectively
12     int entryRow, entryColumn; //entrance coordinates
13     int exitRow, exitColumn; //exit coordinates
14     float cellSize = 2.0f; //size of each cell
15     std::pair<int, int> Entrance;
16     std::pair<int, int> Exit;
17     std::vector<std::vector<Cell>> maze; //accumulates cells to form
        a maze
18     const int dirR[4] = {-1, 0, 1, 0}; //row direction for north,
        east, south, west
19     const int dirC[4] = {0, 1, 0, -1}; //column direction for north,
        east, south, west

```

```

20     int getOppositeWall(int dir) const; //returns the opposite wall
       index
21     public:
22         Maze(int r, int c);
23         void generateMaze(); //generates a maze using DFS
24         Cell& getCell(int r, int c); //returns a reference to a cell
           at (r, c)
25         const Cell& getCell(int r, int c) const; //returns a const
           reference to a cell at (r, c)
26         void printMaze() const; //prints the maze to the console
27         void Draw3D(Model& wallM) const; //for raylib drawing
28         int getRow() const;
29         int getColumn() const;
30         float getCellSize() const;
31         std::pair<int, int> getEntrance() const;
32         std::pair<int, int> getExit() const;
33         std::pair<int, int> worldToCell(Vector3 pos) const;
34         bool isValid(int r, int c) const; //checks if a cell is
           within bounds
35         bool isWall(int r, int c) const;
36 };

```

Listing 14: Maze.hpp — Maze class declaration

A.8 Game.hpp

```

1  #pragma once
2
3  #include <raylib.h>
4  #include "Maze.hpp"
5  #include "Player.hpp"
6  #include "Ghost.hpp"
7  #include "Queue.hpp"
8  #include <vector>
9
10 enum GameState {
11     PLAYING, //1
12     JUMPSCARE, //2
13     LOADING, //3
14     CLEAR, //4
15 };
16
17 class Game {
18     struct Star {
19         Vector3 position;
20         float size, twinkles;
21     };
22     //sounds and music
23     Music permBgSound;
24     Sound exit;
25     Sound jumpscare;
26     bool exitPlayed;
27
28     std::vector<Star> stars;
29     Queue<Ghost> ghosts;
30     int ghostPerLevel;
31     Maze* maze;
32     Player player;

```

```
33     Texture2D grass;
34     Texture2D ghost;
35     GameState state;
36     int mazeSize;
37     int level;
38     float loadingTimer, loadingDuration;
39     Model wallModel;
40     Model grassModel;
41     float jumpscareTimer, jumpscareDuration;
42     bool isAtExit() const;
43     public:
44         Game(Model wall);
45         void Update();
46         void Draw();
47         ~Game();
48 };
```

Listing 15: Game.hpp — Combine everything

B Installation and Usage

The project is available on GitHub at: <https://github.com/asqtecki/Flee-The-Maze>

B.1 Prerequisites

- A C++17-compatible compiler (e.g., g++)
- Make for building the project
- Windows environment with w64devkit (recommended)
- Raylib 5.5 installed at C:/raylib/raylib

B.2 Setup and Compilation

Step 1: Clone the repository

```
1 git clone https://github.com/asqtecki/Flee-The-Maze.git
2 cd Flee-The-Maze
```

Step 2: Configure compiler (Windows only)

```
1 set PATH=C:\raylib\w64devkit\bin;%PATH%
```

Step 3: Build the project

```
1 make clean
2 make
```

B.3 Run the Game

```
1 game.exe
```

Note: Run game.exe from the project root directory to ensure the Graphics/ and Music/ folders are found at their correct relative paths.

B.4 Controls

- **W, A, S, D:** Move the player
- **Shift:** Sprint
- **Mouse:** Look around (first-person view)
- **ESC:** Exit the game

B.5 Gameplay Instructions

- The player starts at the maze entrance.
- Navigate through the maze to reach the exit.
- Each level increases the maze size and difficulty.
- Ghosts use pathfinding to track the player.
- Avoid ghosts and reach the exit to progress.